SE4095 SOCIAL NETWORK THEORY AND PRACTICE · ASSIGNMENT 03

Learning to Seed an Awareness Message with REINFORCE

Budgeted influence maximization on the SocioPatterns workplace contact network with an MLP and a Transformer policy

STUDENT	ROLL NUMBER	CODE	REPRODUCE
M. Waqas Chohan	23F-3041	github.com/Waqas-Chohan/reinforcement-learning	`python main.py --output results`

SUMMARY

An organization may promote a message to only five of 92 employees. I implemented the Independent Cascade simulator, a sequential selection environment and REINFORCE from scratch, and trained a shared-weight MLP policy and a Transformer policy (three seeds each, 1,000 episodes per run). On the same 100 held-out diffusion realizations, Random reached a mean spread of **60.24**, Degree **59.67**, the MLP **57.53 ± 1.39** and the Transformer **58.86 ± 0.65** (mean ± population std over seeds). All 17 verification checks pass. The learned policies stayed close to uniform after 50 updates, so their greedy choices mostly reflect initialization. The main lesson concerns this graph: at p = 0.10 it is supercritical, and five seeds almost always trigger the same large cascade.

1 Dataset and Network Analysis

The instructor-supplied static package derived from *SocioPatterns Contacts in a Workplace* (office building in France, 24 June–3 July 2013) was used unchanged; the SHA-256 hashes of all three CSV files match the dataset README. All 92 participants are loaded from `nodes.csv` before the 755 edges are added. The graph is undirected and unweighted, has no self-loops or duplicate pairs, and aggregates the entire contact period. `results/graph_edges.csv` stores every pair once with source < target. Department labels are used only for figures and for the final descriptive analysis.

Property	Value	Property	Value
Participants	92	Mean degree	16.41
Edges	755	Median / min / max degree	15.5 / 4 / 44
Density	0.1804	Mean local clustering	0.426
Connected components	1	Transitivity	0.368
Isolated participants	0	Diameter / avg. path	3 / 1.96

Table 1. Global properties of the fixed aggregated graph (`results/network_stats.json`).

Rank	Node	Orig. ID	Dept.	Degree	Deg. centr.	Clustering
1	84	804	DMCT	44	0.484	0.256
2	58	311	DSE	38	0.418	0.283
3	14	95	DSE	32	0.352	0.343
4	12	80	DMCT	31	0.341	0.305
5	27	134	DMCT	30	0.330	0.290
6	9	63	SRH	28	0.308	0.251
7	45	223	DSE	28	0.308	0.336
8	24	123	DSE	26	0.286	0.271
9	72	662	DSE	26	0.286	0.403
10	86	826	DISQ	26	0.286	0.323

Table 2. Top-10 participants by degree centrality, degree/(N−1). Ties are broken by the smaller ID.

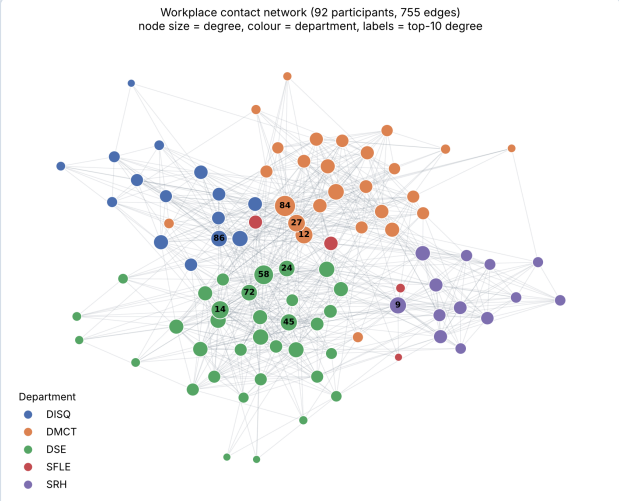


Figure 1. Spring layout of the contact network. Colour shows department, size shows degree, and the top-10 hubs are labelled. Departments form visible clusters, and the hubs (84, 27, 12 in DMCT; 58, 14, 45, 72, 24 in DSE) sit together in the dense core.

The network is small, dense and very compact: every participant is within three hops of every other. Local clustering is high (0.43 on average; 0.17–0.78) and tends to fall as degree rises, so the hubs bridge groups while low-degree members sit inside tight depart-

mental cliques. These hubs are also heavily connected to one another, which is the overlapping-audience problem the assignment describes.

2 Independent Cascade Simulator

I implemented the Independent Cascade Model myself (`environment.py`, `IndependentCascadeSimulator`) using the required live-arc construction. Every undirected edge becomes two directed arcs, giving 1,510 arcs sorted by (source, target). For an episode, `numpy.random.default_rng(diffusion_seed)` draws exactly one uniform number per arc in that order. An arc is *live* if its draw is < 0.10. A breadth-first search from the distinct seeds then counts every participant reachable through live arcs, seeds included. This is equivalent to the "one chance per newly active neighbour" process: each activation attempt is an independent coin flip used at most once, so flipping all coins in advance gives the same distribution of active sets. The same ordered arc list is used for every simulation. Live arcs and draws stay inside the simulator. Policies never see them, and the environment never evaluates hypothetical actions.

Verification check (checks.json)	Result	Verification check (checks.json)	Result
Empty seed set → zero spread	**pass**	Duplicate action raises an error	**pass**
p = 0 → only distinct seeds active (incl. repeated seeds)	**pass**	Exactly five distinct users per episode (also all 800 eval episodes)	**pass**
p = 1 → all users reachable from seeds active	**pass**	Reward = spread / N (unit test and all 800 eval rows)	**pass**
Same graph, seeds and diffusion seed → same spread	**pass**	Masked actions have probability exactly 0 (both policies)	**pass**
Two arcs per undirected edge (1,510 arcs)	**pass**	Finite losses and gradient norms in all 300 updates	**pass**

Table 3. Selected checks. Also passing: out-of-range action rejected, step after termination rejected, N×5 observation and mask, 50 updates per run, 800 evaluation episodes (17 checks in total).

3 Reinforcement-Learning Formulation

State and observation. `reset(diffusion_seed)` fixes the hidden realization for the episode and returns an N×5 matrix in ascending node-ID order together with a Boolean mask (True = unselected). The five features are exactly: degree centrality degree/(N−1); local clustering coefficient; selected indicator; selected-neighbour fraction (selected neighbours / degree, 0 for an isolated node); and remaining budget / 5. The first two are computed once. The last three are recomputed after every action. No department labels, IDs, embeddings, live arcs or future spread are exposed.

Actions and transitions. An action is an integer 0–91 naming an unselected participant. `step(action)` raises `ValueError` for a non-integer, out-of-range or already-selected action instead of silently replacing it. Otherwise it updates the selected set, the features and the mask. **Reward and termination.** Rewards for actions 1–4 are 0. After action 5 the cascade runs once and the episode terminates with R = final spread / 92 $\in [0, 1]$. There is no shaping, penalty or intermediate feedback.

> **WHY THIS IS SEQUENTIAL DECISION-MAKING**
> Each selection removes an option from the action space and changes the selected-indicator, selected-neighbour-fraction and remaining-budget features of the remaining participants. The next choice is therefore conditioned on what has already been chosen. Because the reward depends on the union of what the five seeds reach, the value of a candidate depends on whether its audience is already covered. The objective is a good *combination*, not five independent popularity scores. Training and evaluation use the same graph; only the diffusion realizations differ.

4 REINFORCE Implementation

`reinforce.py` implements REINFORCE (Williams, 1992) in plain PyTorch. During training, actions are sampled from the masked categorical policy, and each step's differentiable log-probability is kept until the update. After 20 complete episodes the loss below is minimized with one Adam step (lr = 0.001), after clipping the global gradient norm at 1.0. With γ = 1 and only a terminal reward, every step of episode i uses the same return R_i:

$$L(\theta) = -\,(1/20)\,\sum_{i=1}^{20}(R_i - b)\,\sum_{t=1}^{5}\log \pi_\theta(a_{i,t} \mid s_{i,t}), \qquad b \leftarrow 0.9b + 0.1\bar{R} \ \text{(after the step)}$$

Returns and b are plain Python floats, so they carry no gradient. b starts at 0 in every run, stays fixed while a batch is collected and its loss computed, and is updated only after the optimizer step. Trajectories are then discarded. Each run therefore makes exactly 50 updates. No critic, entropy bonus, normalization, schedule, replay or early stopping is used. At the start of run $s \in \{7, 21, 42\}$, the Python, NumPy and PyTorch seeds are set to s, and episode e uses diffusion_seed = 1,000,000 + 10,000s + e.

How the update changes behaviour. The gradient of $-(R-b)\log\pi$ pushes up the probability of every action in an episode that beat the baseline, and pushes down every action in an episode that fell short, in proportion to the gap. Over many batches, participants

that often appear in high-spread combinations gain logit mass under similar feature contexts. **Why the baseline helps.** All rewards are positive (about 0.65). Without b, every sampled action would be reinforced and only the size of the push would carry information. Subtracting a running estimate of the typical reward centres the weights, so the update keeps the same expected value (b does not depend on the actions) but has much smaller variance.

5 Policy Architectures

Both policies (`models.py`) output one logit per participant. Selected participants are set to $-\infty$ only at the final stage (`masked_logits`), so the softmax gives them probability exactly 0. Both are trained with identical environment, features, REINFORCE code and settings. **MLP (4,609 parameters):** the same network, 5 → Linear(5,64) → ReLU → Linear(64,64) → ReLU → Linear(64,1), is applied independently to each participant's five features. Any interaction between participants must come through the hand-crafted selection features. **Transformer (33,921 parameters):** N×5 → Linear(5,64) → one `nn.TransformerEncoderLayer` (d_model 64, 4 heads, feed-forward 128, ReLU, dropout 0, post-LayerNorm, default biases) → Linear(64,1) per token. It uses no positional or ID embeddings, no causal mask, no graph-restricted attention and no extra final norm.

- **Tokens.** Each participant is one token whose 64-dimensional embedding is a linear projection of its five features. With no positional embedding the model is permutation-equivariant: a participant is identified only by its features.
- **Queries, keys, values.** Each token is projected into a query q, a key k and a value v. The weight token j receives from token i is $\mathrm{softmax}_j(q_i \cdot k_j/\sqrt{16})$. Token i's output is the weighted average of the values: a context-dependent summary of the other participants.
- **Multi-head attention.** Four heads of size 16 learn separate attention patterns in parallel (for example "attend to selected users" or "attend to hubs"). Their outputs are concatenated and projected back to 64 dimensions.
- **Residual connections and normalization.** Post-norm computes $x \leftarrow \mathrm{LN}(x + \mathrm{Attn}(x))$, then $x \leftarrow \mathrm{LN}(x + \mathrm{FF}(x))$. The residual path keeps each token's own features available, and LayerNorm keeps activations on a stable scale for optimization.
- **Attention vs graph connectivity.** Attention is a learned, fully connected interaction among all 92 tokens and does not follow the contact edges. The graph enters only through the features (degree, clustering, selected-neighbour fraction). Two strangers can attend to each other, and two neighbours may ignore each other.
- **Using selected-user information.** Selected tokens stay visible to attention and carry selected = 1. After each pick, every other token's representation can change according to what has been chosen, even for participants with no link to the selected ones. This gives the Transformer a channel the MLP lacks for learning "complement the current seeds". REINFORCE is the learning algorithm; the Transformer is only the policy architecture.

6 Controlled Evaluation and Results

All six final checkpoints (after episode 1,000) and the two baselines were evaluated on diffusion seeds 900000–900099: exactly 800 episodes with identical realizations for every method. Degree picks the five highest-degree users (ties to the smaller ID). Random uses a separate generator seeded with diffusion_seed + 10,000,000 and samples without replacement. Neural policies choose the highest-probability valid action (exact ties to the smaller ID) with gradients disabled and no updates. Nothing was tuned on evaluation results.

Method	Train seed	Spread mean	Spread std (pop.)	Reward mean	Selection time / episode	Training time
Random	**−1**	**60.24**	**9.72**	**0.6548**	**0.07 ms**	**0**
Degree	−1	59.67	7.26	0.6486	0.06 ms	0
REINFORCE MLP	7	58.88	13.72	0.6400	0.49 ms	2.32 s
REINFORCE MLP	21	58.09	16.11	0.6314	0.71 ms	1.94 s
REINFORCE MLP	42	55.62	18.37	0.6046	0.56 ms	1.92 s
REINFORCE Transformer	7	58.09	16.11	0.6314	1.62 ms	7.87 s
REINFORCE Transformer	21	59.67	7.26	0.6486	1.63 ms	9.66 s
REINFORCE Transformer	42	58.81	14.74	0.6392	1.61 ms	7.83 s

Table 4. `results/summary.csv`. Selection time excludes diffusion. Training time includes all 1,000 training simulations (CPU, 1 thread). Architecture level (mean ± population std of the three run means): **MLP 57.53 ± 1.39**, **Transformer 58.86 ± 0.65**; mean training time 2.06 s vs 8.45 s.

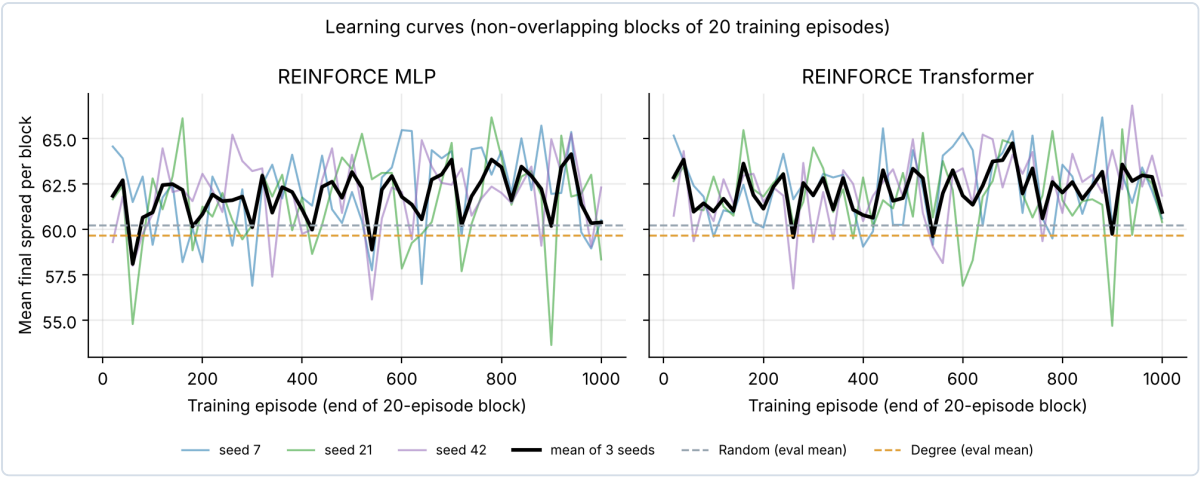


Figure 2. Mean training spread over non-overlapping blocks of 20 episodes (50 blocks per run). Curves stay near 62 with no clear trend. Dashed lines are evaluation means on different realizations, shown only for scale.

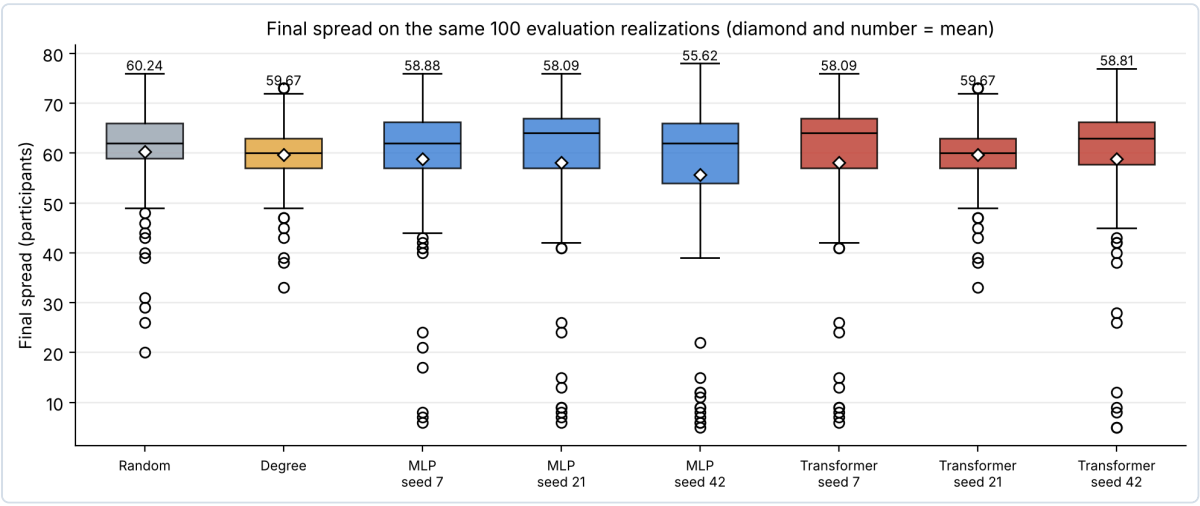


Figure 3. Final spread over the same 100 evaluation realizations. Learned seed sets have higher medians (62–64) but a heavy lower tail: in 4–11 of 100 realizations the cascade dies (< 20 users), which never happens for Degree or Random.

Method (seed)	Selected users (in order)	Departments	Cov.	Σdeg / reach
Degree	84, 58, 14, 12, 27	DMCT DSE DSE DMCT DMCT	2	175 / 75
MLP (7)	66, 0, 36, 82, 85	SRH DSE DISQ DSE DSE	3	47 / 44
MLP (21)	0, 36, 66, 82, 10	DSE DISQ SRH DSE DMCT	4	40 / 43
MLP (42)	13, 71, 26, 89, 17	SFLE DSE DSE DMCT DMCT	3	44 / 36
Transformer (7)	66, 0, 36, 82, 10	SRH DSE DISQ DSE DMCT	4	40 / 43
Transformer (21)	84, 58, 14, 12, 27	DMCT DSE DSE DMCT DMCT	2	175 / 75
Transformer (42)	0, 36, 66, 82, 64	DSE DISQ SRH DSE DSE	3	41 / 38
Random (100 eps.)	different set each episode		3.09	–

Table 5. Selected users and department coverage (distinct departments among seeds; descriptive only). Greedy policies pick the same set for all 100 realizations. Σdeg / reach = sum of seed degrees / size of the union of the seeds' closed neighbourhoods. The Degree seeds share 8 edges among themselves; the learned sets share 0–1.

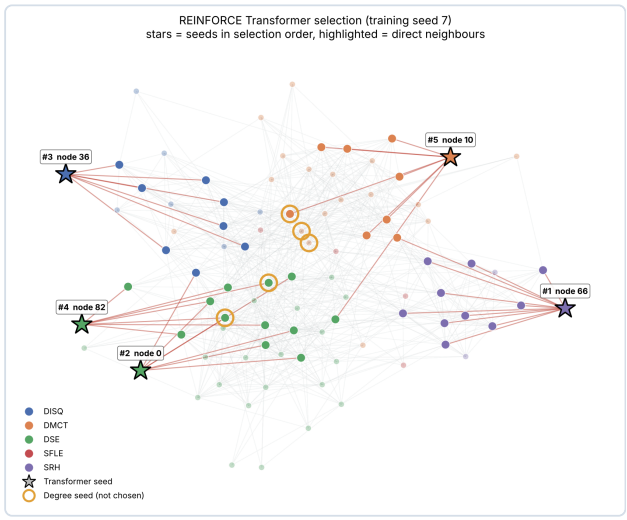


Figure 4. REINFORCE Transformer selection from training seed 7 (stars, numbered in selection order) with the seeds' direct neighbours highlighted. Gold rings mark the Degree seeds the policy did not choose. The learned set is spread across four departments on the periphery, while the Degree seeds cluster in the dense core.

Policy diagnostics (`results/policy_diagnostics.csv`). On the empty selection, the final first-step entropy is 4.518–4.522 against 4.522 for a uniform distribution. The largest probability is at most 0.014, against 0.0109 uniform. In 5 of 6 runs, the top-ranked first pick is the same as at initialization. Transformer seed 21 started and ended strongly correlated with degree (r = 0.92 → 0.99), so its greedy set equals Degree's. The other runs ended with weak or negative correlation to degree, so their greedy sets favour low-degree peripheral participants.

Discussion

Overlapping audiences. The Degree seeds have a combined degree of 175 but reach only 75 distinct participants in one hop, and they share 8 edges, so much of their audience is shared. The learned sets cover only 36–44 users in one hop but overlap almost nowhere. In this graph, however, the overlap problem matters less than expected. The expected number of live out-arcs per node is about $0.10 \times 16.4 \approx 1.6 > 1$, so most realizations contain one giant live component of roughly 60–70 participants. Spread is then mainly "giant component or not": if any seed lands in it, the result is about 62 whatever the seed set. Hubs are almost certain to land in it, which explains Degree's narrow distribution (std 7.3, minimum 33). Low-degree seeds can occasionally all miss it, which explains the learned sets' collapses (minimum 5) and their higher medians when they succeed, because they also start cascades in separate departments. Random sets mix both effects and achieve the highest mean here. The differences between methods (about 1–2 users, ~2% of N) are small relative to the realization noise (std 7–18).

Learning stability. The curves are flat and the policies barely moved from initialization. With rewards concentrated around 0.65, the advantages $R-b$ are small and dominated by realization noise. The largest pre-clip gradient norm per run was only 0.05–0.60, so the 1.0 clip never activated. Fifty Adam steps at lr = 10^{-3} are therefore too few to separate good from bad combinations. The run-to-run spread confirms that seed choice decides the outcome. The Transformer shows somewhat lower run-to-run variability (std 0.65 vs 1.39) and moved further from uniform than the MLP (maximum probability 0.012–0.014 vs 0.011), consistent with attention giving it a richer function class. I do not claim a significant difference from three seeds. **Department coverage.** The learned sets cover 3–4 departments, Degree covers 2 (DMCT and DSE only) and Random 3.09 on average. This is descriptive and was never used for training. **Computational cost.** Training a run takes about 2 s (MLP) and about 8.5 s (Transformer) on one CPU thread. Greedy selection costs 0.5–0.7 ms (MLP) and about 1.6 ms (Transformer) per episode, against about 0.06 ms for the heuristics. The Transformer's attention is $O(N^2)$ per step, which is negligible at N = 92. The entire pipeline runs in about 35 s.

7 Limitations

- All claims apply only to this fixed, aggregated 92-node contact graph. Static aggregation discards contact timing, duration and repetition, so a single brief contact counts the same as daily collaboration.
- Adoption is simulated with Independent Cascade at an assumed p = 0.10. Spread is a model output, not observed message adoption or information-sharing behaviour.
- The protocol fixes 1,000 episodes (50 updates), three seeds and no hyperparameter search. The policies are under-trained, and the results should not be read as the ceiling of RL for this task. With three seeds and 100 realizations, differences of 1–2 users are within noise.
- Training and evaluation share the same graph; only realizations differ. Generalization to other networks is not tested. Department labels were used only for figures and descriptive coverage.
- Reported times are wall-clock times on a 2-vCPU Intel Xeon 2.30 GHz cloud machine (CPU, one PyTorch thread) and vary by about ±20% between runs. Spreads, selections and CSV values are bit-for-bit reproducible.

References

Génois, M., Vestergaard, C. L., Fournet, J., Panisson, A., Bonmarin, I., & Barrat, A. (2015). Data on face-to-face contacts in an office building suggest a low-cost vaccination strategy based on community linkers. *Network Science*, 3(3), 326–347. https://doi.org/10.1017/nws.2015.10

SocioPatterns. *Contacts in a workplace* (2013 release, CC Public Domain Dedication). https://sociopatterns.org/datasets/contacts-in-a-workplace/

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8, 229–256. https://doi.org/10.1007/BF00992696

Kempe, D., Kleinberg, J., & Tardos, É. (2003). Maximizing the spread of influence through a social network. *Proc. KDD*, 137–146.

Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

Zafarani, R., Abbasi, M. A., & Liu, H. (2014). *Social Media Mining: An Introduction*, Ch. 7 (Information Diffusion). Cambridge University Press.

Software: PyTorch 2.15 (`nn.TransformerEncoderLayer`), NumPy 2.5, NetworkX 3.7, pandas 3.0, Matplotlib 3.11. No code was adapted from external RL libraries or pre-trained models.